

 Muhammad Ghafary
Yuhandra

Prediction Model:

Credit Scoring



About Company

Home Credit Indonesia is a consumer-focused financing company that provides a wide range of financial services, from consumer financing to protection. Operating since 2013, Home Credit Indonesia has served more than 6.6 million customers, providing transparent and responsible financial services to help create new opportunities for the community.

What Is The Main Purpose?

Memastikan bahwa
model selalu
menerima
pengajuan kredit
dari pelanggan yang
dianggap mampu



Model dapat
menolak pengajuan
kredit dari
pelanggan yang
tidak mampu
(default)



Memahami
segmentasi
nasabah yang
melakukan
pengajuan kredit



About Dataset

	SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	AMT_CREDIT	AMT_ANNUITY
0	100002	1	Cash loans	M	N	Y	0	202500.0	406597.5	20000.0
1	100003	0	Cash loans	F	N	N	0	270000.0	1293502.5	30000.0
2	100004	0	Revolving loans	M	Y	Y	0	67500.0	135000.0	10000.0
3	100006	0	Cash loans	F	N	Y	0	135000.0	312682.5	20000.0
4	100007	0	Cash loans	M	N	Y	0	121500.0	513000.0	20000.0
...	...	...	...	...	...	...	...	...	...	...
307506	456251	0	Cash loans	M	N	N	0	157500.0	254700.0	20000.0
307507	456252	0	Cash loans	F	N	Y	0	72000.0	269550.0	10000.0
307508	456253	0	Cash loans	F	N	Y	0	153000.0	677664.0	20000.0
307509	456254	1	Cash loans	F	N	Y	0	171000.0	370107.0	20000.0
307510	456255	0	Cash loans	F	N	N	0	157500.0	675000.0	40000.0

307511 rows × 122 columns

Rincian isi dataset:

- 307.511 Baris
- 122 Kolom
- 106 Kolom bertipe data numerik
- 16 Kolom bertipe data kategorikal (object)

```
Jumlah kolom numerik: 106
Jumlah kolom kategorikal: 16
Jumlah kolom boolean: 0

daftar kolom numerik: Index(['SK_ID_CURR', 'TARGET', 'CNT_CHILDREN', 'AMT_INCOME_TOTAL',
                             'AMT_CREDIT', 'AMT_ANNUITY', 'AMT_GOODS_PRICE',
                             'REGION_POPULATION_RELATIVE', 'DAYS_BIRTH', 'DAYS_EMPLOYED',
                             ...,
                             'FLAG_DOCUMENT_18', 'FLAG_DOCUMENT_19', 'FLAG_DOCUMENT_20',
                             'FLAG_DOCUMENT_21', 'AMT_REQ_CREDIT_BUREAU_HOUR',
                             'AMT_REQ_CREDIT_BUREAU_DAY', 'AMT_REQ_CREDIT_BUREAU_WEEK',
                             'AMT_REQ_CREDIT_BUREAU_MON', 'AMT_REQ_CREDIT_BUREAU_QRT',
                             'AMT_REQ_CREDIT_BUREAU_YEAR'],
                             dtype='object', length=106)

daftar kolom kategorikal: Index(['NAME_CONTRACT_TYPE', 'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY',
                                 'NAME_TYPE_SUITE', 'NAME_INCOME_TYPE', 'NAME_EDUCATION_TYPE',
                                 'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'OCCUPATION_TYPE',
                                 'WEEKDAY_APPR_PROCESS_START', 'ORGANIZATION_TYPE', 'FONDKAPREMONT_MODE',
                                 'HOUSETYPE_MODE', 'WALLSMATERIAL_MODE', 'EMERGENCYSTATE_MODE'],
                                 dtype='object')

daftar kolom boolean: Index([], dtype='object')
```

Beberapa jenis kolom berdasarkan informasi yang diberikan :

- ID unik setiap pengajuan pinjaman (SK_ID_CURR)
- Informasi dasar Pinjaman (AMT_CREDIT, AMT_ANNUITY, NAME_CONTRACT_TYPE)
- Profil Klien (CODE_GENDER, AMT_INCOME_TOTAL, OCCUPATION_TYPE)
- Informasi waktu terkait pengajuan (DAYS_BIRTH, DAYS_EMPLOYED, DAYS_LAST_PHONE_CHANGE)
- Skor dari eksternal (EXT_SOURCE_(1, 2, & 3))

Data Cleaning

1. Missing Value

Terdapat 41 kolom yang memiliki missing value di atas 50%, sehingga perlu di-drop

```
def nulls_check(df):
    return (df.isnull().sum()
            .to_frame('null_count')
            .assign(null_ratio=lambda x: x['null_count'] / len(df))
            .query("null_count > 0")
            .sort_values('null_ratio', ascending=False)
            )

high_missing = nulls_check(df)
drop_missing = [col for col in high_missing[high_missing['null_ratio'] > 0.5].index]

print(f'Kolom dengan missing >50%: {len(drop_missing)}')
print(f'Total di-drop: {len(drop_missing)} kolom')
print('\nKolom yang di-drop:')
for c in drop_missing:
    print(f' {c} ({high_missing.loc[c, "null_ratio"]*100:.1f}%)')
```

```
# Drop kolom dengan missing value > 50%
df.drop(columns=drop_missing, inplace=True)
```

Kolom dengan missing >50%: 41
Total di-drop: 41 kolom

Kolom yang di-drop:

COMMONAREA_MEDI (69.9%)
COMMONAREA_MODE (69.9%)
COMMONAREA_AVG (69.9%)
NONLIVINGAPARTMENTS_MODE (69.4%)
NONLIVINGAPARTMENTS_MEDI (69.4%)
NONLIVINGAPARTMENTS_AVG (69.4%)
FONDKAPREMONT_MODE (68.4%)
LIVINGAPARTMENTS_AVG (68.4%)
LIVINGAPARTMENTS_MEDI (68.4%)
LIVINGAPARTMENTS_MODE (68.4%)
FLOORSMIN_MEDI (67.8%)
FLOORSMIN_MODE (67.8%)
FLOORSMIN_AVG (67.8%)
YEARS_BUILD_MODE (66.5%)
YEARS_BUILD_MEDI (66.5%)
YEARS_BUILD_AVG (66.5%)
OWN_CAR_AGE (66.0%)
LANDAREA_AVG (59.4%)
LANDAREA_MEDI (59.4%)
LANDAREA_MODE (59.4%)

Adapun untuk kolom dengan missing value dengan jumlah $\leq 50\%$ maka perlu di-impute sesuai dengan pola missing yang terdapat dalam dataset seperti pada gambar di samping. Berikut beberapa penjelasan di antaranya:

1. AMT_REQ_CREDIT_BUREAU_MON/YEAR yang missing memiliki default rate yang lebih tinggi daripada tidak missing karena terdapat kemungkinan ID dari yang mengajukan belum dilakukan pengecekan oleh biro kredit sehingga diberikan nilai 0 serta membuat kolom flag baru untuk ID yang memiliki missing tersebut.
2. EXT_SOURCE_3 juga serupa dengan nomor 1 namun diisi dengan nilai median. Nilai yang missing dalam kolom ini juga mengindikasikan adanya risiko karena default rate untuk nilai yang missing lebih tinggi dibandingkan jika tidak.
3. Kolom yang missingnya memiliki default rate lebih rendah (seperti AMT_ANNUITY) diisi dengan dapat diisi dengan nilai median
4. Kolom OCCUPATION_TYPE diisi dengan value 'Unknown'



```
# — 1. OCCUPATION_TYPE & NAME_TYPE_SUITE → kategori 'Unknown'
df['OCCUPATION_TYPE'] = df['OCCUPATION_TYPE'].fillna('Unknown')
df['NAME_TYPE_SUITE'] = df['NAME_TYPE_SUITE'].fillna('Unknown')

# — 2. EXT_SOURCE_3 → median + flag missing
df['EXT_SOURCE_3_MISSING'] = df['EXT_SOURCE_3'].isnull().astype(int)
df['EXT_SOURCE_3'] = df['EXT_SOURCE_3'].fillna(df['EXT_SOURCE_3'].median())

# — 3. AMT_REQ_CREDIT_BUREAU → 0 + flag missing
df['IS_NO_BUREAU_ENQUIRY'] = df['AMT_REQ_CREDIT_BUREAU_MON'].isnull().astype(int)
df['AMT_REQ_CREDIT_BUREAU_MON'] = df['AMT_REQ_CREDIT_BUREAU_MON'].fillna(0)
df['AMT_REQ_CREDIT_BUREAU_YEAR'] = df['AMT_REQ_CREDIT_BUREAU_YEAR'].fillna(0)

# — 4. DEF_30 & OBS_30 → 0
df['DEF_30_CNT_SOCIAL_CIRCLE'] = df['DEF_30_CNT_SOCIAL_CIRCLE'].fillna(0)
df['OBS_30_CNT_SOCIAL_CIRCLE'] = df['OBS_30_CNT_SOCIAL_CIRCLE'].fillna(0)

# — 5. MCAR → median
for col in ['EXT_SOURCE_2', 'AMT_GOODS_PRICE', 'AMT_ANNUITY', 'DAYS_LAST_PHONE_CHANGE']:
    df[col] = df[col].fillna(df[col].median())
```

2. Redundant and Low Varians Column

Menghapus seluruh kolom redundan yang berakhiran AVG, MODE, dan MEDI serta seluruh kolom FLAG_DOCUMENT yang memiliki varians mendekati nol

```
# Drop triplet redundan _AVG/_MODE/_MEDI
redundant_cols = [col for col in df.columns if '_AVG' in col or '_MODE' in col or '_MEDI' in col]
df.drop(columns=redundant_cols, inplace=True)

# Drop seluruh FLAG_DOCUMENT
drop_flag_doc = ['FLAG_DOCUMENT_2', 'FLAG_DOCUMENT_3', 'FLAG_DOCUMENT_4', 'FLAG_DOCUMENT_5',
                 'FLAG_DOCUMENT_6', 'FLAG_DOCUMENT_7', 'FLAG_DOCUMENT_8', 'FLAG_DOCUMENT_9',
                 'FLAG_DOCUMENT_10', 'FLAG_DOCUMENT_11', 'FLAG_DOCUMENT_12', 'FLAG_DOCUMENT_13',
                 'FLAG_DOCUMENT_14', 'FLAG_DOCUMENT_15', 'FLAG_DOCUMENT_16', 'FLAG_DOCUMENT_17',
                 'FLAG_DOCUMENT_18', 'FLAG_DOCUMENT_19', 'FLAG_DOCUMENT_20', 'FLAG_DOCUMENT_21']
df.drop(columns=drop_flag_doc, inplace=True)
```


3. Very Low and Very High Correlation Columns

Menghapus kolom-kolom yang memiliki korelasi sangat rendah dengan kolom TARGET (warna kuning) serta kolom yang memiliki nilai korelasi yang sangat tinggi dengan kolom lainnya (warna hijau dan merah tua)

Kotak berwarna kuning menunjukkan tidak adanya atau rendahnya korelasi antar kolom, sedangkan kotak berwarna hijau dan merah menunjukkan terdapat korelasi baik positif ataupun negatif.

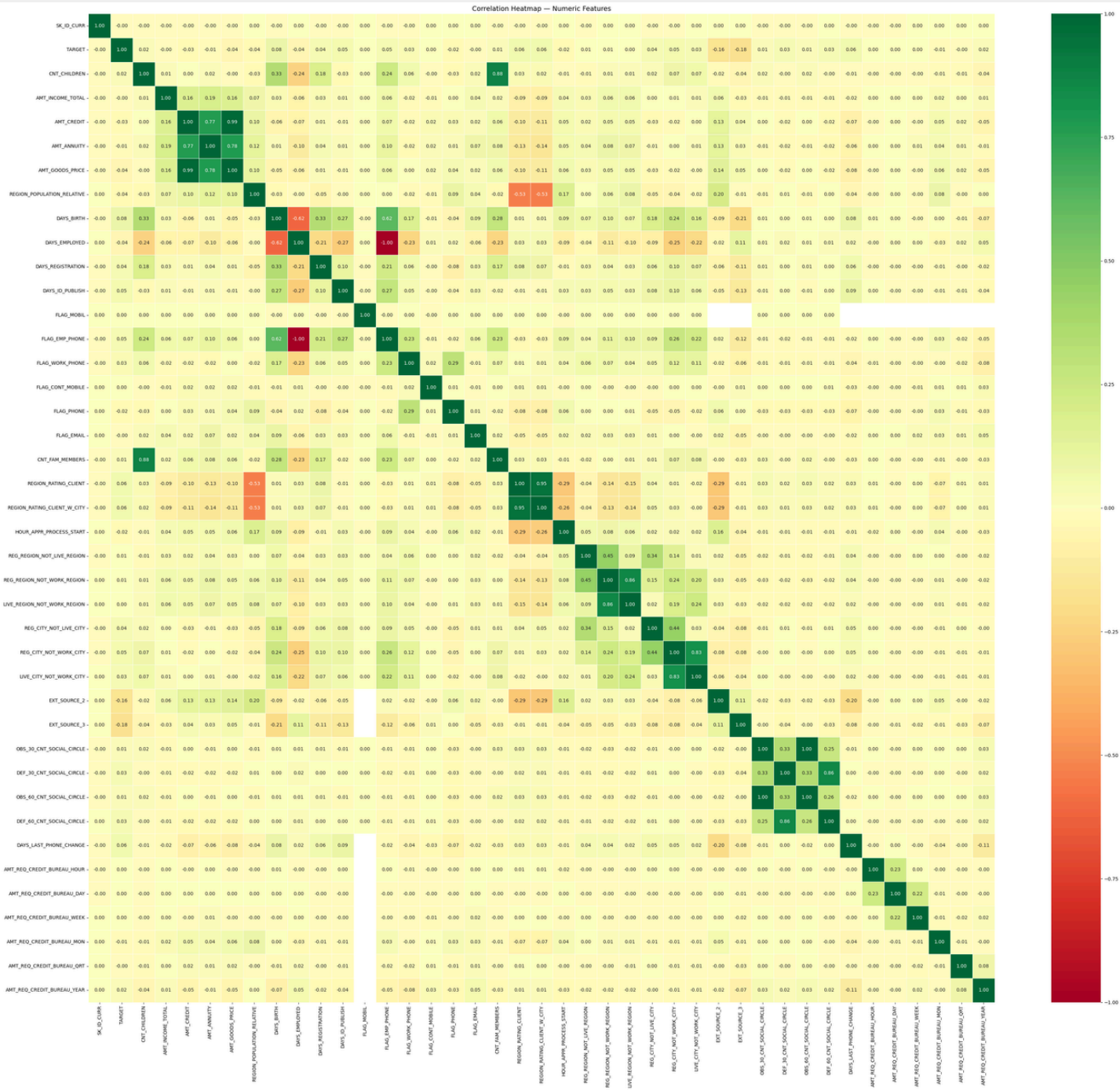
Kini, jumlah kolom yang tersisa setelah dilakukan drop column hanya sebanyak 36 kolom

```
# Drop low corr dengan TARGET
drop_lowcorr = ['SK_ID_CURR', 'FLAG_CONT_MOBILE', 'FLAG_MOBIL', 'FLAG_EMAIL',
               'AMT_REQ_CREDIT_BUREAU_HOUR', 'AMT_REQ_CREDIT_BUREAU_WEEK',
               'AMT_REQ_CREDIT_BUREAU_DAY', 'AMT_REQ_CREDIT_BUREAU_QRT',
               'LIVE_REGION_NOT_WORK_REGION', 'REG_REGION_NOT_LIVE_REGION', 'REG_REGION_NOT_WORK_REGION']
df.drop(columns=drop_lowcorr, inplace=True)

# Drop multikolinearitas tinggi
drop_highcorr = ['DAYS_EMPLOYED', 'OBS_60_CNT_SOCIAL_CIRCLE', 'AMT_CREDIT',
                'REGION_RATING_CLIENT', 'CNT_FAM_MEMBERS', 'DEF_60_CNT_SOCIAL_CIRCLE']
df.drop(columns=drop_highcorr, inplace=True)

print(f'Shape final: {df.shape}')
```

Shape final: (307511, 36)

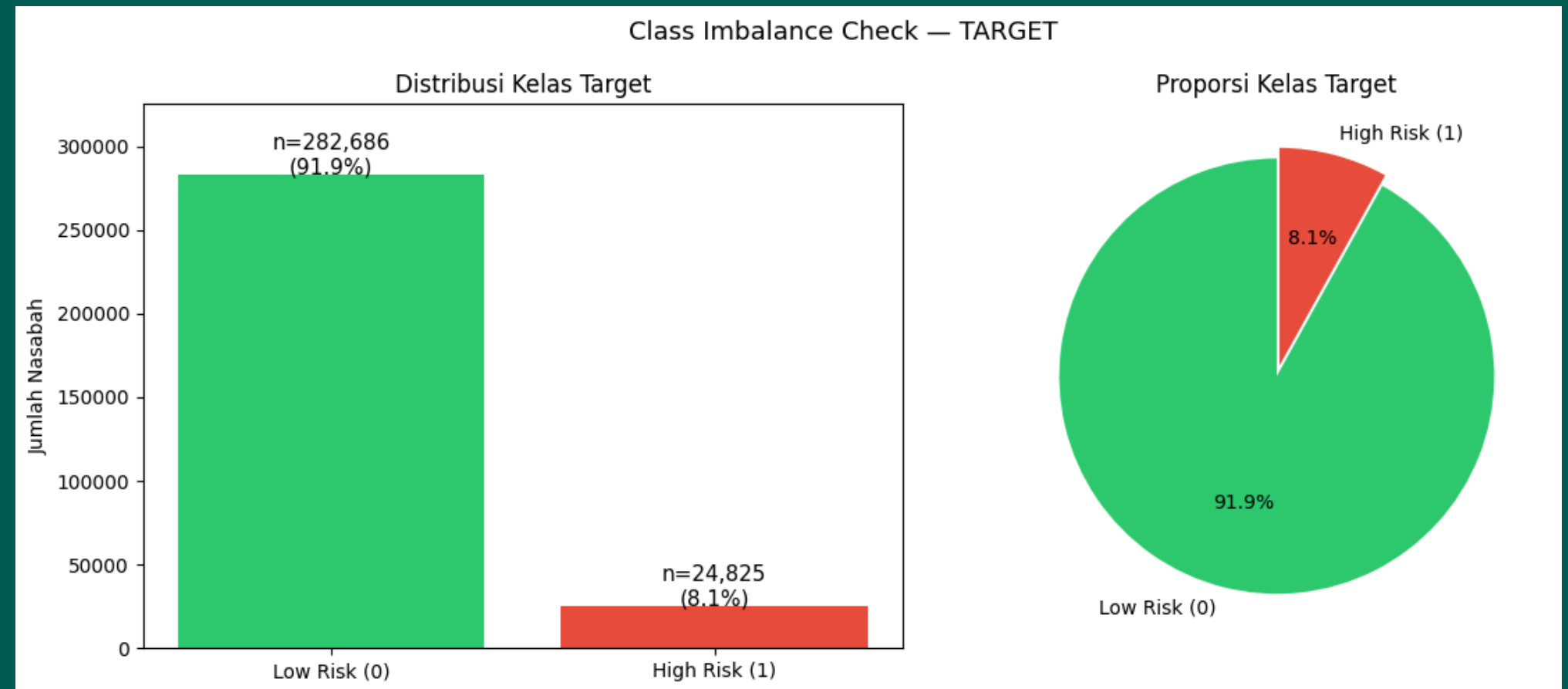


Exploratory Data Analysis

Target Column Distribution

Hasil pengecekan kelas target menunjukkan bahwa terdapat imbalance yang sangat buruk antara low_risk (**91.9%**) dan high_risk (hanya **8.1%**). Jika dibuatkan rasio, maka perbandingannya low dan high risk adalah **11.4 : 1**

Rasio imbalance yang tinggi akan berpengaruh dalam performa model dalam memprediksi fitur target. Untuk mengatasi dapat melakukan tindakan **class_weight = 'balanced'** saat inisiasi model. Hal ini dilakukan dengan cara memberikan penalti yang lebih tinggi pada kelas minoritas (high_risk) sehingga kedua kelas target dianggap setara



Numeric Column Distributions

Berdasarkan sebaran nilainya, kolom-kolom numerik berikut dikelompokkan ke dalam beberapa jenis:

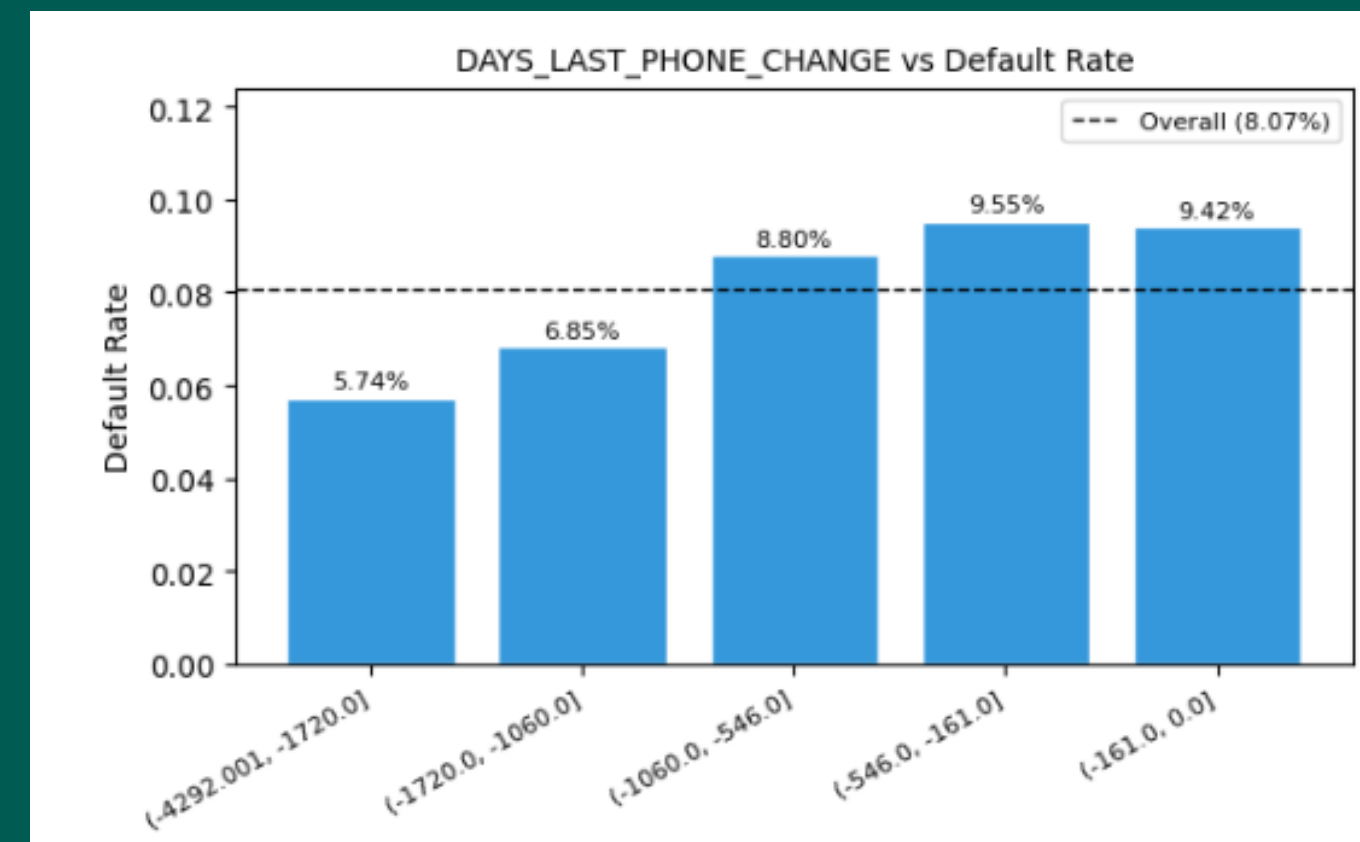
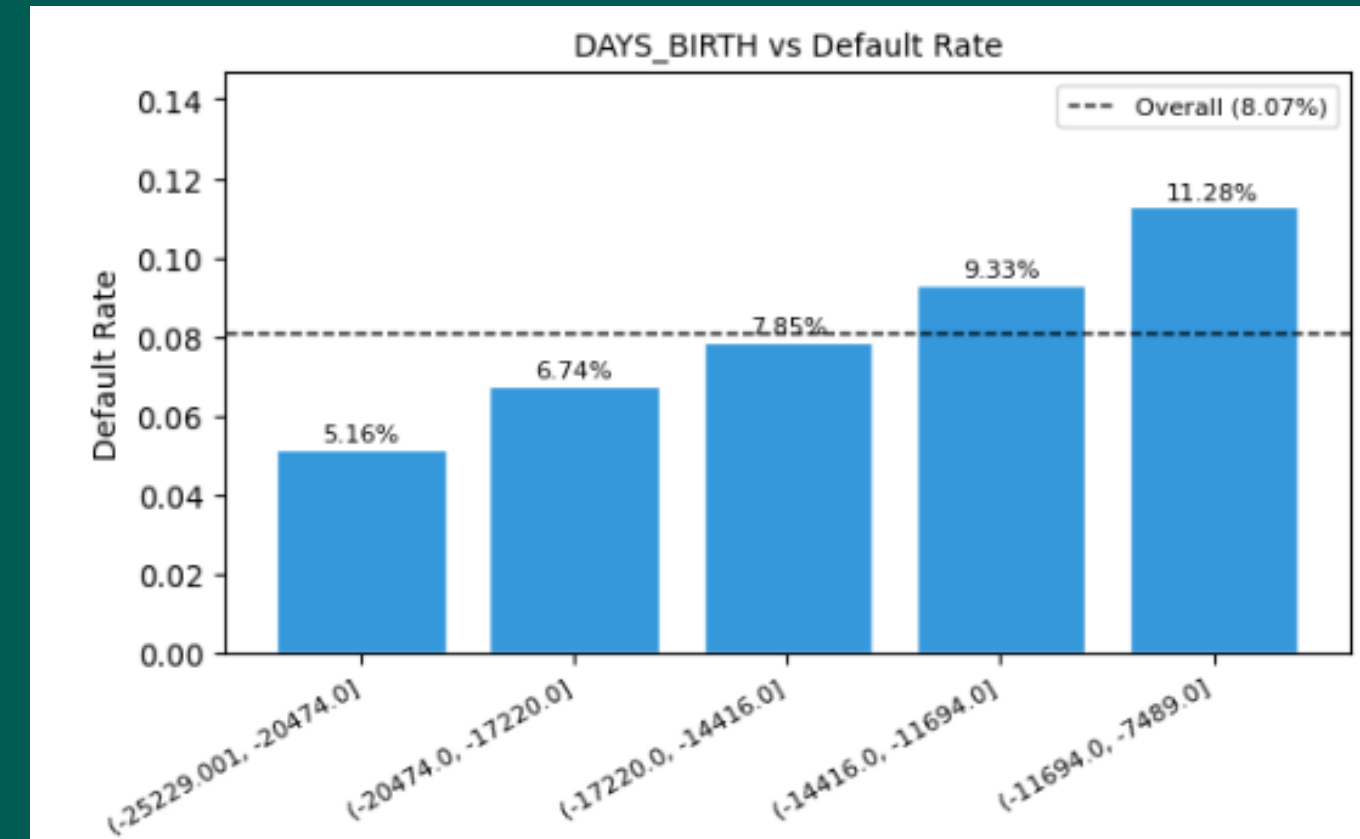
1. **Distribusi Beta**, distribusi nilai yang hanya berada pada rentang tertentu, seperti 0-1. Dapat digunakan untuk mengamati proporsi atau probabilitas
2. **Distribusi Gamma**, dapat digunakan pada nilai positif dan right-skewed. Biasanya digunakan untuk mengamati waktu atau jumlah kejadian peristiwa
3. **Distribusi Normal**, distribusi yang simetris yang menandakan bahwa sebagian besar data terkumpul di sekitar nilai mean
4. **Distribusi Eksponensial**, sebagian besar data terkumpul pada nilai yang paling kecil (kiri). Semakin ke kanan nilai tersebut semakin tinggi namun frekuensinya berkurang
5. **Distribusi Lognormal**, Terdapat pada distribusi data yang right-skewed yang jika diambil nilai logaritmanya maka akan membentuk distribusi normal
6. **Distribusi Uniform**, frekuensi data untuk semua kelompok nilai bersifat seragam

	feature	best_distribution	sse
0	TARGET	beta	9.118304e+02
1	CNT_CHILDREN	expon	6.122651e-01
2	AMT_INCOME_TOTAL	uniform	1.790306e-13
3	AMT_ANNUITY	beta	2.680464e-11
4	AMT_GOODS_PRICE	gamma	4.293595e-12
5	REGION_POPULATION_RELATIVE	beta	1.065753e+04
6	DAYS_BIRTH	beta	2.658340e-09
7	DAYS_REGISTRATION	beta	6.575668e-09
8	DAYS_ID_PUBLISH	lognorm	2.513933e-07
9	FLAG_EMP_PHONE	norm	1.709791e+03
10	FLAG_WORK_PHONE	gamma	1.281984e+03
11	FLAG_PHONE	gamma	7.803526e+02
12	REGION_RATING_CLIENT_W_CITY	beta	3.513096e+02
13	HOURL_APPR_PROCESS_START	beta	2.245084e-01
14	REG_CITY_NOT_LIVE_CITY	beta	9.740185e+02
15	REG_CITY_NOT_WORK_CITY	gamma	9.109479e+02
16	LIVE_CITY_NOT_WORK_CITY	gamma	1.124039e+03
17	EXT_SOURCE_2	beta	3.834814e+00
18	EXT_SOURCE_3	beta	1.118710e+02
19	OBS_30_CNT_SOCIAL_CIRCLE	norm	5.369096e-04
20	DEF_30_CNT_SOCIAL_CIRCLE	norm	2.418540e-01
21	DAYS_LAST_PHONE_CHANGE	beta	2.552929e-06
22	AMT_REQ_CREDIT_BUREAU_MON	expon	6.514756e-02
23	AMT_REQ_CREDIT_BUREAU_YEAR	beta	3.036949e-01

Numeric Features vs High Risk (Default) Rate

Setelah dikelompokkan, data tersebut kemudian visualisasi untuk mengamati tingkat High risk pada masing-masing fitur. Berikut adalah beberapa temuan penting dalam data tersebut:

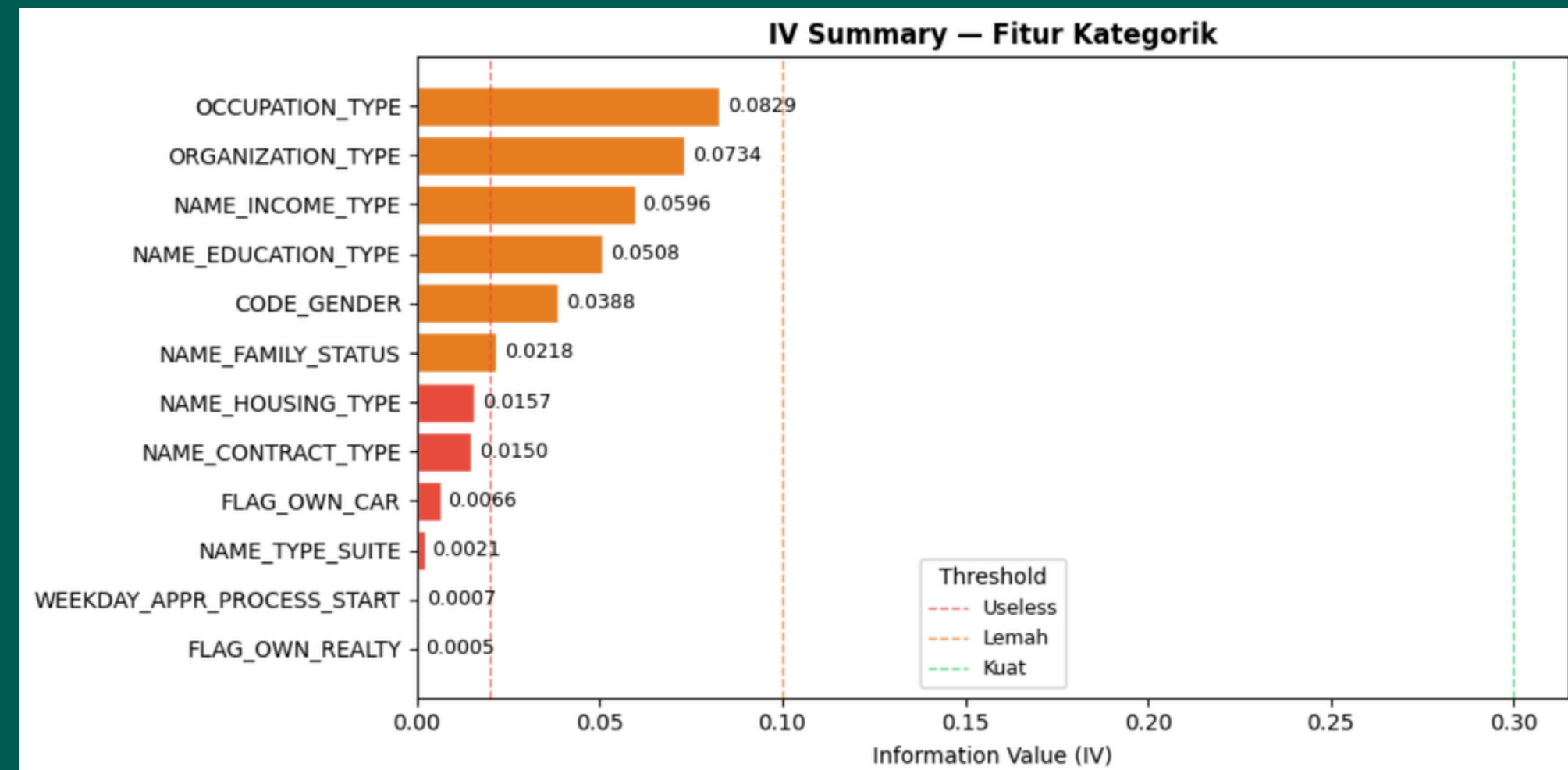
1. Pada fitur **DAYS_BIRTH**, dapat diamati bahwa, semakin **muda** umur klien maka **peluang defaultnya semakin tinggi** (hingga mencapai 11.28% untuk umur 20-32 tahun. Begitu pula sebaliknya untuk umur 50-69 tahun yang hanya mencapai 5.16%.
2. Pada fitur **DAYS_LAST_PHONE_CHANGE**, ditemukan bahwa terdapat pola kenaikan default rate yang signifikan. Klien yang telah lama tidak mengganti nomor HP selama lebih dari 4.5 tahun hanya memiliki default rate sebesar 5.74%. Terjadi kenaikan cukup tajam bagi klien yang telah mengganti nomor HP nya dalam kurang dari 2 tahun terakhir (8.8% - 9.4%).



Information Value (IV) of Categorical Features

Information Value (IV) adalah metrik yang digunakan untuk mengukur tingkat kekuatan suatu fitur dalam membedakan dua kelas target (High/Low Risk). Fitur ini lazim digunakan dalam fitur berjenis kategorikal. Semakin tinggi nilai IV menandakan bahwa fitur tersebut semakin kuat untuk membedakan kedua kelas target.

Pada gambar di samping, IV terbesar hanyalah sebesar 0.08 yang dipegang oleh fitur `OCCUPATION_TYPE` sehingga dapat disimpulkan bahwa seluruh fitur kategorikal tersebut dapat dikatakan lemah. Meskipun demikian, selama fitur-fitur tersebut memiliki nilai di atas 0.02, maka tetap dapat digunakan sebagai prediktor untuk kelas target.

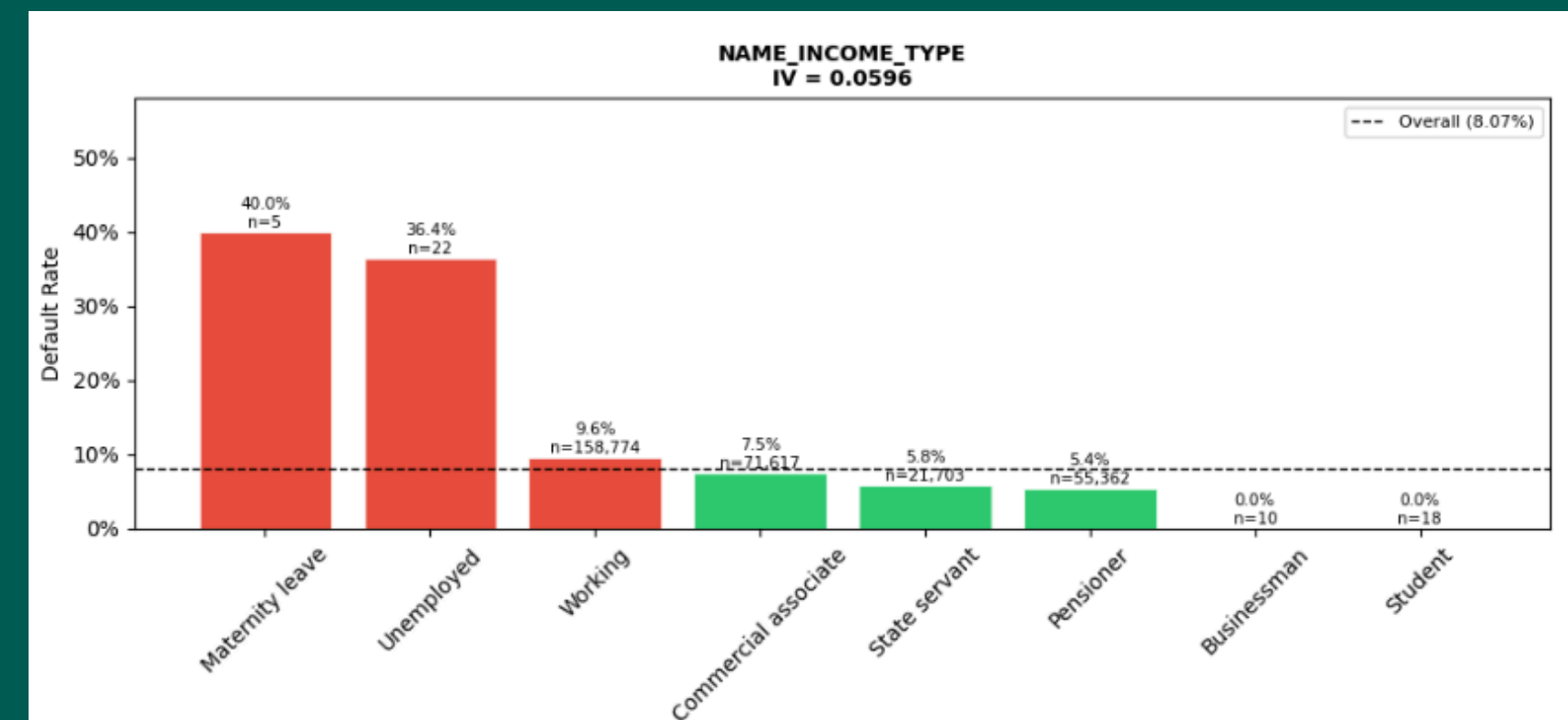
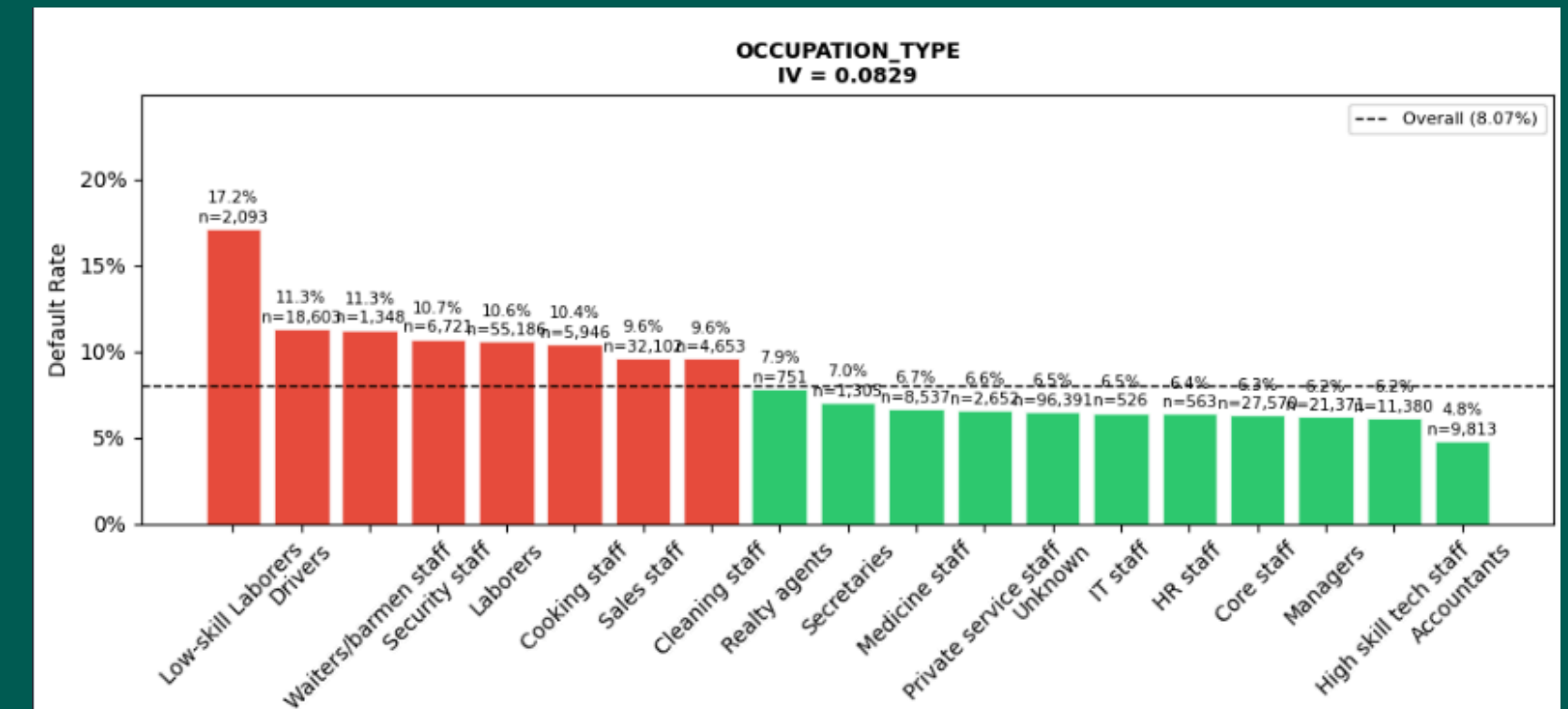


Categorical Features vs High Risk (Default) Rate

Berikut adalah beberapa temuan penting mengenai tingkat default pada fitur kategorikal yang telah dianalisis menggunakan Information Value:

1. Fitur **OCCUPATION_TYPE** merupakan fitur dengan IV tertinggi (0.083) di antara fitur kategorikal lainnya. Pekerjaan kasar seperti low-skill laborers memiliki rate yang tinggi di antara pekerjaan lainnya (17.32%), lalu diikuti oleh pekerjaan drivers (13.3%) dengan selisih cukup besar hingga 4%

2. Fitur **NAME_INCOME_TYPE** memiliki pola yang unik. Klien yang sedang menjalani cuti melahirkan justru memiliki tingkat default yang lebih tinggi (40%) dibandingkan dengan yang tidak memiliki pekerjaan (36%). Adapun jenis pendapatan dengan kategori bekerja memiliki tingkat default hanya mencapai 9.6% dan berada di urutan ketiga setelah dua jenis pendapatan sebelumnya



Exploring Data with Logit Regression

Logit Regression adalah teknik eksplorasi untuk memahami bagaimana hubungan antara suatu variabel prediktor dengan kelas target. Teknik ini digunakan pada fitur numerik. Berikut adalah beberapa temuan penting pada analisis dengan logit:

1. Fitur dengan nilai P-value di atas 0.05 menandakan bahwa fitur tersebut signifikan dan secara statistik dan memiliki pengaruh kuat terhadap data, seperti CNT_CHILDREN, REGION_POPULATION_RELATIVE, REG_CITY_NOT_WORK_CITY
2. Fitur EXT_SOURCE_2/3 memiliki nilai koefisien yang mirip (-2). fitur ini menunjukkan arah log-odds negatif yang artinya fitur ini cenderung menurunkan risiko default. Selain itu kedua fitur ini juga memiliki standard error yang kecil sehingga fitur tersebut memiliki koefisien yang lebih stabil

Logit Regression Results						
=====						
Dep. Variable:	TARGET	No. Observations:	307511			
Model:	Logit	Df Residuals:	307480			
Method:	MLE	Df Model:	30			
Date:	Mon, 01 Jun 2026	Pseudo R-squ.:	0.08917			
Time:	00:57:58	Log-Likelihood:	-78578.			
converged:	True	LL-Null:	-86271.			
Covariance Type:	nonrobust	LLR p-value:	0.000			
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-0.3230	0.076	-4.222	0.000	-0.473	-0.173
CNT_CHILDREN	0.0005	0.019	0.026	0.979	-0.037	0.038
AMT_INCOME_TOTAL	-2.374e-09	1.86e-08	-0.128	0.898	-3.88e-08	3.41e-08
AMT_ANNUITY	1.124e-05	7.4e-07	15.190	0.000	9.79e-06	1.27e-05
AMT_GOODS_PRICE	-4.622e-07	3.18e-08	-14.519	0.000	-5.25e-07	-4e-07
REGION_POPULATION_RELATIVE	-0.6435	0.630	-1.021	0.307	-1.879	0.592
DAYS_BIRTH	1.144e-05	2.16e-06	5.284	0.000	7.19e-06	1.57e-05
DAYS_REGISTRATION	7.541e-06	2.18e-06	3.464	0.001	3.27e-06	1.18e-05
DAYS_ID_PUBLISH	3.013e-05	4.82e-06	6.252	0.000	2.07e-05	3.96e-05
FLAG_EMP_PHONE	0.1928	0.027	7.250	0.000	0.141	0.245
FLAG_WORK_PHONE	0.1448	0.018	8.195	0.000	0.110	0.179
FLAG_PHONE	-0.1200	0.017	-7.100	0.000	-0.153	-0.087
REGION_RATING_CLIENT_W_CITY	0.1659	0.016	10.337	0.000	0.134	0.197
HOURL_APPR_PROCESS_START	-0.0073	0.002	-3.362	0.001	-0.012	-0.003
REG_CITY_NOT_LIVE_CITY	0.1780	0.034	5.309	0.000	0.112	0.244
REG_CITY_NOT_WORK_CITY	0.0252	0.038	0.668	0.504	-0.049	0.099
LIVE_CITY_NOT_WORK_CITY	0.0664	0.036	1.822	0.068	-0.005	0.138
EXT_SOURCE_2	-2.2321	0.035	-63.547	0.000	-2.301	-2.163
EXT_SOURCE_3	-2.8486	0.040	-71.139	0.000	-2.927	-2.770
OBS_30_CNT_SOCIAL_CIRCLE	0.0059	0.004	1.539	0.124	-0.002	0.013
DEF_30_CNT_SOCIAL_CIRCLE	0.0841	0.030	2.785	0.005	0.025	0.143
DAYS_LAST_PHONE_CHANGE	8.291e-05	9.32e-06	8.892	0.000	6.46e-05	0.000
AMT_REQ_CREDIT_BUREAU_MON	-0.0224	0.014	-1.636	0.102	-0.049	0.004
AMT_REQ_CREDIT_BUREAU_YEAR	0.0147	0.005	2.993	0.003	0.005	0.024
EXT_SOURCE_3_MISSING	0.0875	0.030	2.937	0.003	0.029	0.146
IS_NO_BUREAU_ENQUIRY	0.4080	0.035	11.584	0.000	0.339	0.477
DEF_30_CNT_SOCIAL_CIRCLE_flag	0.1592	0.044	3.617	0.000	0.073	0.245
OBS_30_CNT_SOCIAL_CIRCLE_flag	-0.0542	0.019	-2.834	0.005	-0.092	-0.017
CNT_CHILDREN_flag	0.0089	0.031	0.289	0.772	-0.051	0.069
AMT_REQ_CREDIT_BUREAU_MON_flag	-0.0151	0.029	-0.521	0.602	-0.072	0.042
AMT_REQ_CREDIT_BUREAU_YEAR_flag	0.0584	0.022	2.677	0.007	0.016	0.101
=====						

Feature Engineering & Selection

Feature Selection

Berdasarkan hasil tahapan Exploratory Data Analysis (EDA) yang telah dilakukan sebelumnya, selanjutnya dilakukan seleksi fitur yang akan digunakan dalam model machine learning dengan pertimbangan:

1. Fitur dengan nilai IV di bawah 0.01 karena dari nilai tersebut, sebuah fitur tidak mampu untuk membedakan dua kelas target
2. Fitur dengan pola atau selisih default rate yang default yang tidak signifikan
3. Fitur dengan nilai P-value tidak signifikan (≥ 0.05)

```
# STEP 1 – DROP FITUR TIDAK SIGNIFIKAN
# =====
# Dasar keputusan:
#   Kategorik : IV < 0.01 (useless)
#   Numerik   : selisih default rate kecil + p-value > 0.05 dari logit diagnostic

# -- Kategorik (IV terlalu kecil) --
drop_cat = [
    'FLAG_OWN_CAR',          # IV = 0.007, selisih hanya 1.3%
    'FLAG_OWN_REALTY',      # IV = 0.007, selisih hanya 0.3%
    'NAME_TYPE_SUITE',      # IV = 0.002, hampir noise
    'WEEKDAY_APPR_PROCESS_START', # IV = 0.0007, hari pengajuan tidak prediktif
]

# -- Numerik (selisih kecil + tidak signifikan di logit) --
drop_num = [
    'CNT_CHILDREN',          # p=0.979, selisih +1.2% saja
    'CNT_CHILDREN_flag',     # p=0.772, turunannya juga tidak signifikan
    'REGION_POPULATION_RELATIVE', # p=0.307, pola zig-zag di EDA
    'REG_CITY_NOT_WORK_CITY',  # p=0.504, tidak signifikan
    'LIVE_CITY_NOT_WORK_CITY', # p=0.068, borderline + interpretasi lemah
    'OBS_30_CNT_SOCIAL_CIRCLE', # p=0.124, selisih hanya +0.43%
    'OBS_30_CNT_SOCIAL_CIRCLE_flag', # turunannya juga lemah
    'AMT_REQ_CREDIT_BUREAU_MON', # p=0.102, arah terbalik + tidak signifikan
    'AMT_REQ_CREDIT_BUREAU_MON_flag', # p=0.602
]

all_drop = [c for c in drop_cat + drop_num if c in df.columns]
df.drop(columns=all_drop, inplace=True)

print(f'Shape setelah drop: {df.shape}')
print(f'Kolom di-drop: {len(all_drop)}')
```

Feature Engineering

Feature engineering adalah tahapan transformasi data menjadi menjadi prediktor yang relevan dengan kebutuhan bisnis dan informatif. Tahapan ini juga bertujuan agar model dapat mempelajari data secara efektif sehingga menghasilkan performa yang lebih akurat. Berikut adalah beberapa teknik feature engineering yang digunakan:

1. Mengganti format pada fitur bertipe waktu yang sebelumnya berupa hitungan hari menjadi tahun agar lebih mudah diinterpretasi
2. Membuat fitur baru yang lebih informatif seperti keuangan dan stabilitas klien
3. Mengagregasi dua fitur, EXT_SOURCE_2&3

```
# — 2A. Transformasi Waktu (hari → tahun, nilai absolut) —
# DAYS_* bernilai negatif → jadikan positif dan ubah ke tahun

df['AGE_YEARS'] = (-df['DAYS_BIRTH'] / 365).round(1)
df['REGISTRATION_YEARS'] = (-df['DAYS_REGISTRATION'] / 365).round(1)
df['ID_PUBLISH_YEARS'] = (-df['DAYS_ID_PUBLISH'] / 365).round(1)
df['PHONE_CHANGE_YEARS'] = (-df['DAYS_LAST_PHONE_CHANGE'] / 365).round(1)

# Drop kolom DAYS asli — sudah tergantikan oleh versi tahun
df.drop(columns=['DAYS_BIRTH', 'DAYS_REGISTRATION',
                  'DAYS_ID_PUBLISH', 'DAYS_LAST_PHONE_CHANGE'], inplace=True)

print('✓ Fitur waktu: AGE_YEARS, REGISTRATION_YEARS, ID_PUBLISH_YEARS, PHONE_CHANGE_YEARS')

# — 2B. Rasio Keuangan —
# Rasio lebih informatif dari nilai absolut karena sudah normalize per klien

# Hindari pembagian dengan 0
eps = 1e-9

# Beban cicilan relatif terhadap pendapatan
df['ANNUITY_INCOME_RATIO'] = df['AMT_ANNUITY'] / (df['AMT_INCOME_TOTAL'] + eps)

# Harga barang relatif terhadap pendapatan
df['GOODS_INCOME_RATIO'] = df['AMT_GOODS_PRICE'] / (df['AMT_INCOME_TOTAL'] + eps)

# Cicilan relatif terhadap harga barang (proxy LTV / down payment)
df['ANNUITY_GOODS_RATIO'] = df['AMT_ANNUITY'] / (df['AMT_GOODS_PRICE'] + eps)

print('✓ Rasio keuangan: ANNUITY_INCOME_RATIO, GOODS_INCOME_RATIO, ANNUITY_GOODS_RATIO')

# — 2C. Agregasi EXT_SOURCE —
# EXT_SOURCE_2 dan EXT_SOURCE_3 adalah fitur terkuat
# Kombinasikan untuk menangkap sinyal gabungan

df['EXT_SOURCE_MEAN'] = df[['EXT_SOURCE_2', 'EXT_SOURCE_3']].mean(axis=1)
df['EXT_SOURCE_MIN'] = df[['EXT_SOURCE_2', 'EXT_SOURCE_3']].min(axis=1)
df['EXT_SOURCE_PROD'] = df['EXT_SOURCE_2'] * df['EXT_SOURCE_3']

print('✓ Agregasi EXT_SOURCE: EXT_SOURCE_MEAN, EXT_SOURCE_MIN, EXT_SOURCE_PROD')

# — 2D. Indikator Ketidakstabilan —
# Gabungkan sinyal ketidakcocokan alamat menjadi skor tunggal

df['ADDRESS_MISMATCH_SCORE'] = df['REG_CITY_NOT_LIVE_CITY'].astype(int)

# Skor stabilitas dokumen — makin lama tidak ganti = lebih stabil
# Encode: baru ganti (< 1 tahun) = 1 (tidak stabil), sudah lama = 0
df['RECENTLY_CHANGED_ID'] = (df['ID_PUBLISH_YEARS'] < 1).astype(int)
df['RECENTLY_CHANGED_PHONE'] = (df['PHONE_CHANGE_YEARS'] < 1).astype(int)

print('✓ Indikator stabilitas: ADDRESS_MISMATCH_SCORE, RECENTLY_CHANGED_ID, RECENTLY_CHANGED_PHONE')
```


4. Menangani data dengan distribusi skewed dengan log-transform agar data berdistribusi lebih normal

5. Membatasi nilai maksimum pada data dengan cara outlier capping untuk menghindari nilai ekstrem yang dapat berpengaruh saat scaling nanti

6. Encoding seluruh fitur kategorikal. Fitur yang memiliki hanya 2 kategori (0/1) menggunakan label encoding sedangkan fitur yang memiliki lebih dari 2 kategori menggunakan one-hot encoding

7. Transformasi data numerik dengan normalisasi (scaling), yaitu menjadikan data bernilai dengan rentang mulai dari 0 hingga 1 agar data menjadi seragam

```
# — 2E. Log Transform untuk Distribusi Skewed —————
# Khusus untuk Logistic Regression agar distribusi lebih normal
# Tidak mempengaruhi LightGBM (tree-based tidak butuh ini)
# Gunakan log1p untuk menangani nilai 0 dengan aman

df['LOG_AMT_INCOME_TOTAL'] = np.log1p(df['AMT_INCOME_TOTAL'])
df['LOG_AMT_ANNUITY']      = np.log1p(df['AMT_ANNUITY'])
df['LOG_AMT_GOODS_PRICE'] = np.log1p(df['AMT_GOODS_PRICE'])

# Drop versi original – sudah ada versi log
# Catatan: versi original masih digunakan di rasio (2B), jadi drop setelah ini
df.drop(columns=['AMT_INCOME_TOTAL', 'AMT_ANNUITY', 'AMT_GOODS_PRICE'], inplace=True)

print('✓ Log transform: LOG_AMT_INCOME_TOTAL, LOG_AMT_ANNUITY, LOG_AMT_GOODS_PRICE')

# — 2F. Outlier Capping (sebelum scaling) —————
# Cap di persentil 99 untuk fitur dengan outlier ekstrem

cap_cols = {
    'DEF_30_CNT_SOCIAL_CIRCLE': 99, # max=34
    'AMT_REQ_CREDIT_BUREAU_YEAR': 99, # max=25
    'ANNUITY_INCOME_RATIO': 99, # bisa sangat besar jika income kecil
    'GOODS_INCOME_RATIO': 99,
    'ANNUITY_GOODS_RATIO': 99,
}

for col, pct in cap_cols.items():
    if col in df.columns:
        cap_val = df[col].quantile(pct / 100)
        df[col] = df[col].clip(upper=cap_val)
        print(f' Cap {col} at p{pct}: {cap_val:.4f}')

print('✓ Outlier capping selesai')
```

```
# STEP 1 – ENCODING KATEGORIK
# =====
cat_cols      = df.select_dtypes('object').columns.tolist()
binary_cats   = [c for c in cat_cols if df[c].nunique() == 2]
nonbinary_cats = [c for c in cat_cols if df[c].nunique() > 2]

print(f'Kolom kategorik   : {len(cat_cols)}')
print(f' Binary (LE)      : {binary_cats}')
print(f' Non-binary (OHE): {nonbinary_cats}')

# Label Encoding untuk binary
le = LabelEncoder()
for col in binary_cats:
    df[col] = le.fit_transform(df[col].astype(str))

# One-Hot Encoding untuk non-binary
df = pd.get_dummies(df, columns=nonbinary_cats, drop_first=True)

# Konversi bool → float
bool_cols = df.select_dtypes('bool').columns
df[bool_cols] = df[bool_cols].astype(float)

# STEP 4 – SCALING
# =====
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train),
    columns=feature_cols
)
X_val_scaled = pd.DataFrame(
    scaler.transform(X_val),
    columns=feature_cols
)

print('✓ StandardScaler selesai (fit di train, transform di val)')
```

Model 1: Logistic Regression

Logistic Regression adalah model klasifikasi dalam machine learning yang memprediksi fitur kategorikal (ya/tidak, 0/1) berdasarkan satu atau lebih variabel prediktor. Model ini bekerja dengan menggunakan fungsi sigmoid yang mengubah output menjadi probabilitas antara 0 dan 1 lalu diberikan batas pemisah antara keduanya dengan threshold. Model ini sering dijadikan sebagai baseline karena algoritmanya sederhana, cepat, dan mudah diinterpretasi.

Sebelum menjalankan model, data di-split menjadi data train dan data validation dengan proporsi 80:20 agar model dapat dievaluasi terlebih dahulu sebelum dijalankan pada data test

Parameter awal yang digunakan dalam model:

- C : 0.1
- penalty : l2
- class_weight : balanced
- solver : lbfgs
- max_iter : 1000

Threshold Tuning

Sesuai dengan kebutuhan bisnis di awal, yaitu selalu menerima pengajuan kredit dari klien yang mampu, maka metrik yang sebaiknya perlu diperhatikan adalah:

- 1. **Specificity (True Negative Rate)**, memastikan klien yang benar-benar mampu tidak ditolak oleh model. Semakin tinggi nilainya maka semakin baik
- 2. **False Positive Rate**, merupakan kebalikan dari specificity, yaitu mengurangi sebanyak mungkin peluang nasabah mampu yang tertolak
- 3. **Recall**, semakin banyak klien yang diterima maka model juga akan berpeluang menerima klien yang default. Karena itu, threshold tuning perlu dilakukan untuk menyeimbangkan trade-off antara klien yang harus diterima dan yang harus ditolak
- 4. **AUC-ROC**, untuk mengukur seberapa baik model dalam membedakan dua kelas target, semakin mendekati 1 nilai maka dianggap semakin baik

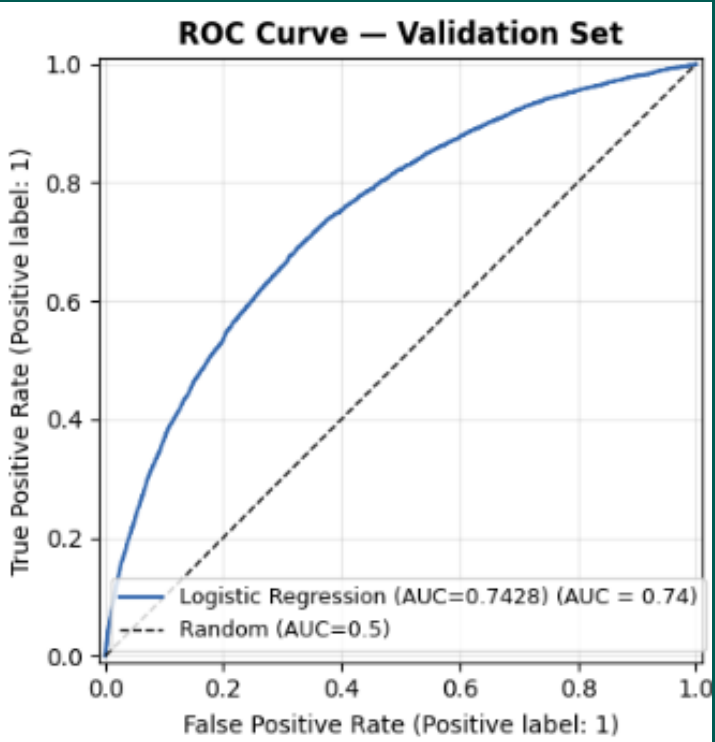
Thresh	Specificity	FPR	Salah Tolak (FP)	Recall	Lolos Default (FN)	Precision
0.10	0.0131	98.69%	55,796	0.9970	15	0.0815
0.15	0.0609	93.91%	53,096	0.9889	55	0.0846
0.20	0.1351	86.49%	48,897	0.9720	139	0.0898
0.25	0.2263	77.37%	43,742	0.9476	260	0.0971
0.30	0.3253	67.47%	38,144	0.9108	443	0.1060
0.35	0.4236	57.64%	32,587	0.8647	672	0.1164
0.40	0.5167	48.33%	27,327	0.8115	936	0.1285
0.45	0.6047	39.53%	22,347	0.7490	1,246	0.1427
0.50	0.6844	31.56%	17,841	0.6779	1,599	0.1587
0.55	0.7568	24.32%	13,752	0.5936	2,018	0.1765
0.60	0.8178	18.22%	10,303	0.5112	2,427	0.1976
0.65	0.8714	12.86%	7,273	0.4228	2,866	0.2240
0.70	0.9150	8.50%	4,807	0.3303	3,325	0.2544
0.75	0.9490	5.10%	2,886	0.2344	3,801	0.2874
0.80	0.9740	2.60%	1,469	0.1509	4,216	0.3377
0.85	0.9900	1.00%	568	0.0717	4,609	0.3853
0.90	0.9979	0.21%	119	0.0171	4,880	0.4167

THRESHOLD OPTIMAL: 0.6
Syarat minimum Recall : 50%
Specificity : 0.8178
FPR (salah tolak) : 18.22%
Recall (tangkap default): 0.5112
Precision : 0.1976

Confusion Matrix (threshold=0.6):

	Pred: Disetujui	Pred: Ditolak
Aktual: Mampu Bayar	46,235	10,303
Aktual: Default	2,427	2,538

- ✓ Klien mampu bayar BENAR disetujui : 46,235 (81.78%)
- ✗ Klien mampu bayar SALAH ditolak (FP) : 10,303 (18.22%)
- ✓ Klien default BENAR ditolak : 2,538 (51.12%)
- ⚠ Klien default LOLOS disetujui (FN) : 2,427 (48.88%)



Cross Validation

Untuk mengetahui konsistensi model sebelum diujikan, maka perlu dilakukan teknik cross validation, yaitu membagi data ke dalam beberapa subset, misalnya dibagi atas 5 subset yg terdiri atas 4 bagian data train dan 1 bagian dataset kemudian diujikan 5 kali bergantian.

Nilai evaluasi dari setiap iterasi tersebut kemudian dihitung nilai rata-ratanya dan standar deviasnya. Semakin kecil nilai standar deviasnya maka model tersebut dianggap objektif dan tidak bias atau overfitting

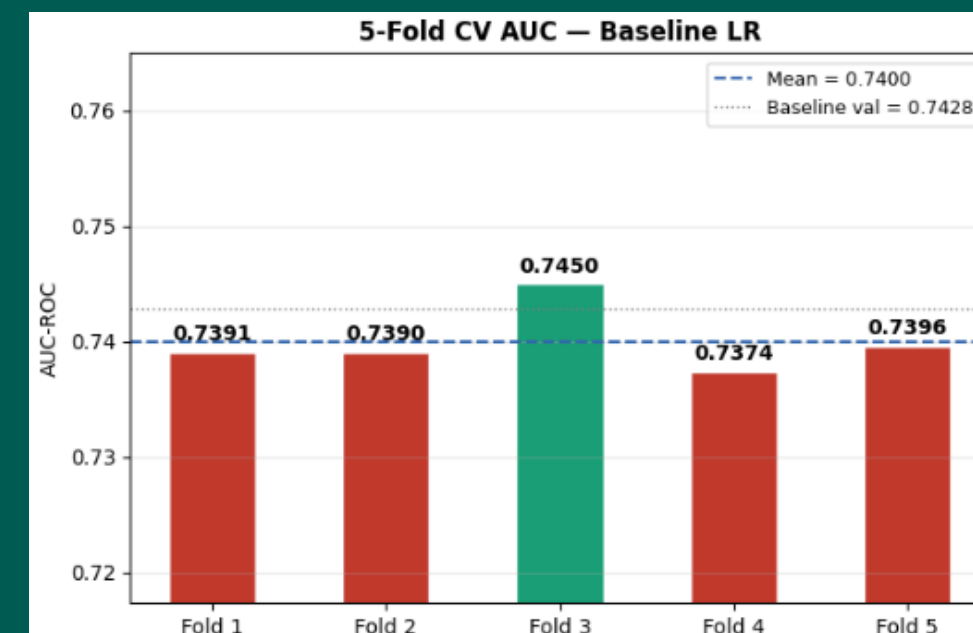
```
=====
STEP 1 - CROSS VALIDATION BASELINE (5-Fold Stratified)
=====
Menggunakan: X_train_scaled & y_train
Val set tetap terpisah untuk evaluasi final

AUC per fold:
Fold 1: 0.7391
Fold 2: 0.7390
Fold 3: 0.7450
Fold 4: 0.7374
Fold 5: 0.7396

Rata-rata AUC : 0.7400
Std deviasi   : 0.0026
Min           : 0.7374
Max           : 0.7450
Range (max-min): 0.0076

✅ Model SANGAT STABIL - std < 0.005, aman untuk di-tuning

Baseline val AUC sebelumnya : 0.7428
Baseline CV AUC sekarang    : 0.7400
Selisih                     : 0.0028
```



Hyperparameter Tuning

Hyperparameter tuning adalah tahapan dalam machine learning yang mengatur terkait cara model ‘belajar’ dari sebuah data. Beberapa hyperparameter yang umum digunakan dalam model klasifikasi adalah:

1. **C**, yaitu parameter yang mengatur seberapa kuat regularisasi model. semakin besar nilai C maka model semakin longgar dan berpotensi overfitting, vice versa
2. **Penalty**, berbeda dengan C, penalty mengatur cara menghukum model ‘menghukum’ bobot koefisien yang terlalu besar. Jenis penalty yang sering digunakan adalah L1 (memangkas bobot hingga nol) dan L2 (mengecilkan bobot mendekati nol)
3. **Max iteration**, mengatur iterasi model hingga mendapatkan nilai koefisien yang optimal (konvergen)
4. **Class weight**, untuk menangani imbalanced data

```
STEP 2 - HYPERPARAMETER TUNING (GridSearchCV + 5-Fold CV)
=====
Parameter grid:
C: [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0]
penalty: ['l2']
solver: ['lbfgs']

Total kombinasi: 8 kombinasi x 5 fold
Harap tunggu...

Hasil semua kombinasi (sorted by Val AUC):
  C | Penalty | Train AUC | Val AUC | Std | Gap
-----
  1.0 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
  5.0 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
  0.5 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
 10.0 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
  0.1 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
  0.05 | 12 | 0.7423 | 0.7400 | 0.0026 | 0.0023
  0.01 | 12 | 0.7420 | 0.7397 | 0.0027 | 0.0023
 0.001 | 12 | 0.7408 | 0.7387 | 0.0027 | 0.0021

✓ Parameter terbaik : {'C': 1.0, 'penalty': 'l2', 'solver': 'lbfgs'}
✓ Best CV AUC : 0.7400
Baseline CV AUC : 0.7400
Peningkatan : +0.0000
```

```
=====
RINGKASAN FINAL - LOGISTIC REGRESSION
=====
Parameter      : C=1.0, penalty=l2, class_weight=balanced
CV AUC          : 0.7400 ± 0.0026
Val AUC         : 0.7428
Threshold       : 0.6

Dari 56,538 klien MAMPU BAYAR:
→ 46,228 (81.76%) BENAR disetujui ✓
→ 10,310 (18.24%) SALAH ditolak ✗

Dari 4,965 klien DEFAULT:
→ 2,538 (51.12%) BENAR ditolak ✓
→ 2,427 (48.88%) LOLOS disetujui ⚠
```

Evaluasi dan Hasil Akhir Logistic Regression

Model 2: Light GBM

LightGBM adalah model machine learning berbasis Gradient Boosting, yaitu membangun decision tree secara berurutan dan setiap tree baru belajar dari kesalahan tree sebelumnya. Model ini disebut 'Light' karena tumbuhnya secara vertikal dengan mencari daun dengan nilai error terbesar (leaf-wise growth). Model ini dianggap cocok untuk data yang berukuran sangat banyak.

Seperti halnya dengan model Logistic Regression, data di-split menjadi data train dan data validation dengan proporsi 80:20 agar model dapat dievaluasi terlebih dahulu sebelum dijalankan pada data test.

```
lgb_baseline = lgb.LGBMClassifier(  
    n_estimators      = 500,  
    learning_rate     = 0.05,  
    num_leaves        = 31,  
    max_depth         = -1,  
    min_child_samples = 20,  
    subsample         = 0.8,  
    colsample_bytree  = 0.8,  
    class_weight      = 'balanced',  
    random_state       = RANDOM,  
    n_jobs            = -1,  
    verbose           = -1  
)
```

Parameter awal untuk model LightGBM

Cross Validation

Data kembali dibagi menjadi 5 bagian bagian (subset) dan secara bergantian setiap fold menjadi data test untuk mengamati konsistensi model LightGBM.

Hasil cross validation menunjukkan bahwa model sangat stabil mempelajari setiap fold dengan nilai standar deviasi yang sangat kecil.

```
cv_scores = cross_val_score(
    lgb_baseline, X_train, y_train,
    cv=skf, scoring='roc_auc', n_jobs=-1
)

print('AUC per fold:')
for i, s in enumerate(cv_scores, 1):
    bar = '█' * int(s * 40)
    print(f'  Fold {i}: {s:.4f} {bar}')
print(f'\nRata-rata AUC : {cv_scores.mean()}')
print(f'Std deviasi    : {cv_scores.std()}')
if cv_scores.std() < 0.005:
    print('✅ Model SANGAT STABIL')
elif cv_scores.std() < 0.01:
    print('✅ Model STABIL')
else:
    print('⚠️ Model KURANG STABIL')
```

```
=====
STEP 1 - BASELINE LIGHTGBM + 5-FOLD CV
=====
AUC per fold:
Fold 1: 0.7434
Fold 2: 0.7465
Fold 3: 0.7478
Fold 4: 0.7459
Fold 5: 0.7482

Rata-rata AUC : 0.7464
Std deviasi   : 0.0017
✔ Model SANGAT STABIL

[100] valid_0's binary_logloss: 0.583281
[200] valid_0's binary_logloss: 0.572116
[300] valid_0's binary_logloss: 0.564295
[400] valid_0's binary_logloss: 0.556806
[500] valid_0's binary_logloss: 0.550163

Baseline Val AUC : 0.7527
LR Val AUC       : 0.7428 (pembanding)
```


Hyperparameter Tuning

Kemudian, melakukan hyperparameter tuning dengan menggunakan library 'Optuna' agar model dapat belajar lebih efisien, berikut adalah beberapa parameter yang biasanya di-tuning dalam model LightGBM:

1. **n_estimators**, yaitu jumlah tree (pohon) yang dibuat.
2. **learning_rate**, mengukur seberapa besar update setiap tree
3. **max_depth**, yaitu tingkat kedalaman sebuah tree
4. **num_leaves**, mengontrol jumlah daun (leaf) setiap tree
5. **min_child_samples**, minimal jumlah data dalam leaf
6. **subsamples**, jumlah sampling data yang diambil pada setiap tree
7. **colsample_bytree**, jumlah fitur yang diambil setiap tree
8. **reg_alpha** (regularisasi L1) dan **reg_lambda** (regularisasi L2)

```
def objective(trial):
    params = {
        'n_estimators' : trial.suggest_int('n_estimators', 100, 500),
        'learning_rate' : trial.suggest_float('learning_rate', 0.01, 0.15, log=True),
        'num_leaves' : trial.suggest_int('num_leaves', 20, 80),
        'max_depth' : trial.suggest_int('max_depth', 4, 20),
        'min_child_samples': trial.suggest_int('min_child_samples', 10, 50),
        'subsample' : trial.suggest_float('subsample', 0.7, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.7, 1.0),
        'reg_alpha' : trial.suggest_float('reg_alpha', 0.01, 0.5, log=True),
        'reg_lambda' : trial.suggest_float('reg_lambda', 0.01, 0.5, log=True),
    }

    model = lgb.LGBMClassifier(
        **params,
        class_weight = 'balanced',
        random_state = RANDOM,
        n_jobs = -1,
        verbose = -1
    )

    scores = cross_val_score(
        model, X_train, y_train,
        cv=skf, scoring='roc_auc', n_jobs=-1
```

```
    return scores.mean()
```

```
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=30, show_progress_bar=True)
```

```
best_params = study.best_params
best_cv_auc = study.best_value
```

STEP 2 - HYPERPARAMETER TUNING (Optuna, 30 trials)
Menggunakan 5-Fold CV di dalam setiap trial

Best trial: 23. Best value: 0.749581: 100% 30/30 [1:14:12<00:00, 130.57s/it]

✓ Best CV AUC : 0.7496
Baseline CV AUC: 0.7464
Peningkatan : +0.0032

Parameter terbaik:

n_estimators	: 302
learning_rate	: 0.02929000481679277
num_leaves	: 33
max_depth	: 13
min_child_samples	: 41
subsample	: 0.910008563888559
colsample_bytree	: 0.7817067367374476
reg_alpha	: 0.08410734272644305
reg_lambda	: 0.03893329714803651

Threshold Tuning

Seperti halnya dengan model Logistic Regression sebelumnya model dievaluasi dengan metrik yang serupa (AUC-ROC, Specificity, FPR, dan Recall) serta dilakukan threshold tuning untuk mendapatkan nilai evaluasi terbaik untuk seluruh metrik

Hasil tuning menunjukkan bahwa nilai threshold optimal untuk model LightGBM adalah 0.6 dengan nilai metrik:

Specificity: 0.8222

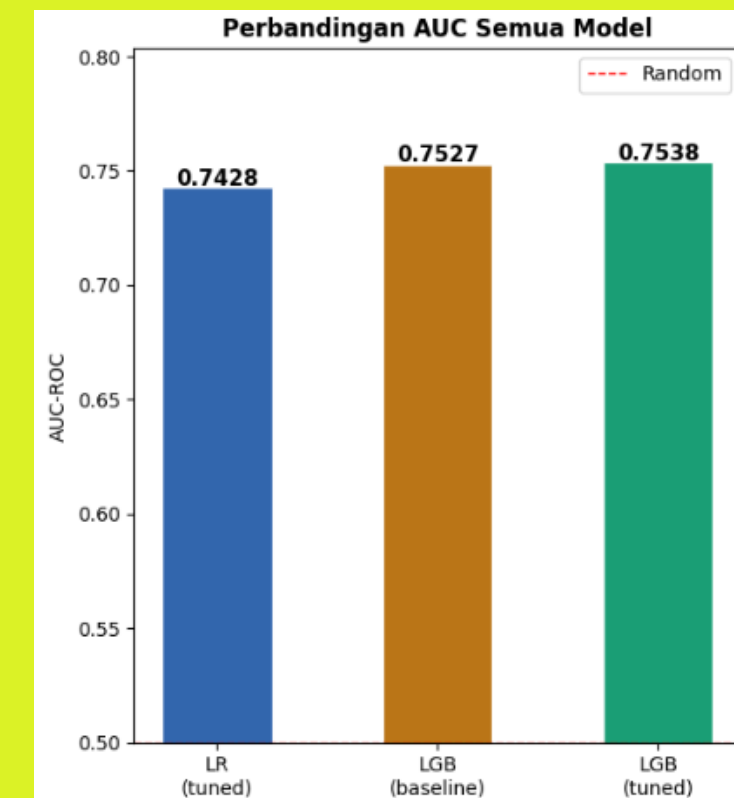
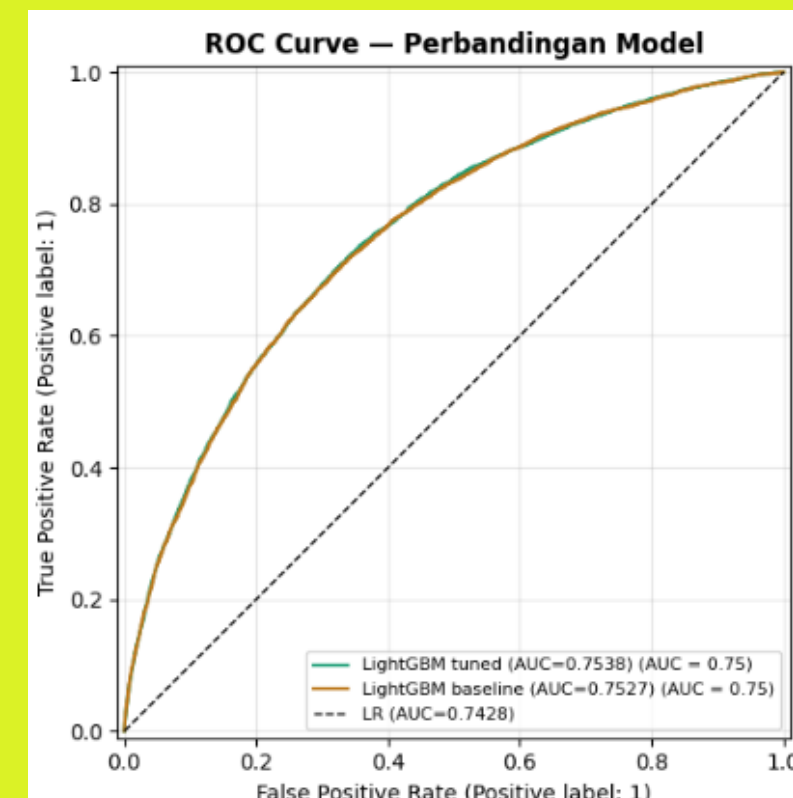
FPR: 17.78%

Recall: 0.5219

AUC-ROC: 0.7538

Thresh	Specificity	FPR	Salah Tolak (FP)	Recall	Lolos Default (FN)	Precision
0.10	0.0218	97.82%	55,306	0.9978	11	0.0822
0.15	0.0753	92.47%	52,280	0.9885	57	0.0858
0.20	0.1640	83.60%	47,263	0.9688	155	0.0924
0.25	0.2688	73.12%	41,339	0.9384	306	0.1013
0.30	0.3737	62.63%	35,409	0.8969	512	0.1117
0.35	0.4705	52.95%	29,935	0.8568	711	0.1244
0.40	0.5564	44.36%	25,082	0.8024	981	0.1371
0.45	0.6324	36.76%	20,785	0.7436	1,273	0.1508
0.50	0.7015	29.85%	16,879	0.6759	1,609	0.1659
0.55	0.7639	23.61%	13,349	0.6016	1,978	0.1828
0.60	0.8222	17.78%	10,055	0.5219	2,374	0.2049
0.65	0.8738	12.62%	7,137	0.4332	2,814	0.2316
0.70	0.9164	8.36%	4,726	0.3394	3,280	0.2628
0.75	0.9531	4.69%	2,649	0.2421	3,763	0.3121
0.80	0.9796	2.04%	1,151	0.1364	4,288	0.3704
0.85	0.9945	0.55%	312	0.0516	4,709	0.4507
0.90	0.9997	0.03%	16	0.0052	4,939	0.6190

✅ Threshold optimal : 0.6
Specificity : 0.8222
FPR : 17.78%
Recall : 0.5219



Perbandingan akhir skor AUC-ROC model Logistic Regression dan LightGBM

Comparing Logistic Regression and LightGBM Performance

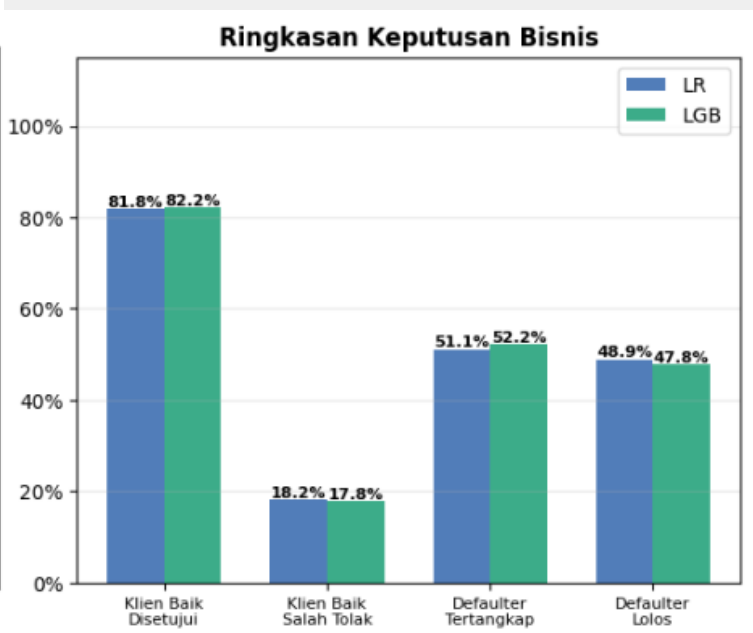
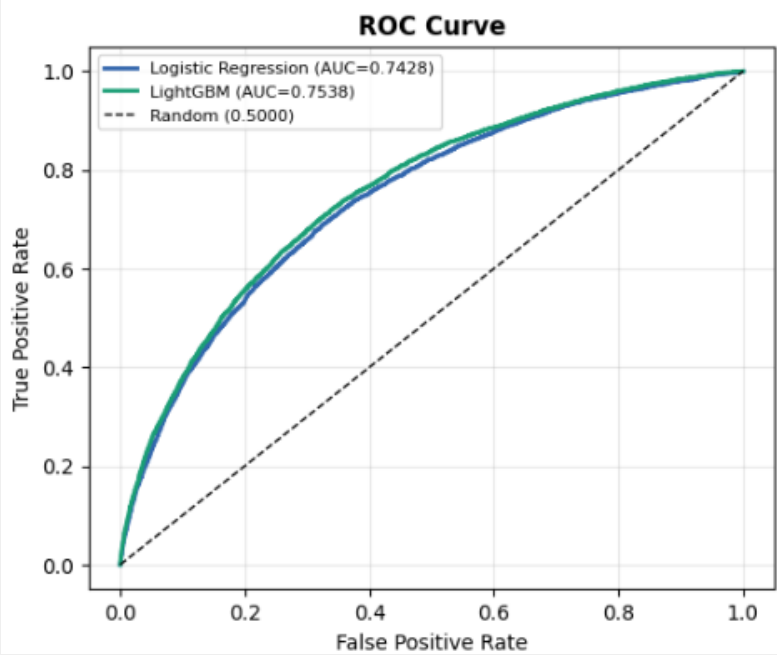
Berdasarkan hasil prediksi pada data validation, Model LightGBM unggul dalam seluruh metrik evaluasi. LightGBM lebih banyak menerima pengajuan dari klien yang mampu (Specificity) sebanyak 0.82 dengan jumlah klien 46.483, dibandingkan Logistic regression yang menerima lebih sedikit 248 klien dari LightGBM.

Selain itu, LightGBM juga lebih baik dalam salah menolak klien mampu (FPR) dengan nilai 0.1778, lebih unggul 0.0044 dari Logistic Regression (LR). LightGBM juga memiliki nilai AUC yang lebih tinggi dibandingkan LR.

Dengan demikian, dapat disimpulkan bahwa **model LightGBM secara keseluruhan memprediksi klien yang mampu dan tidak mampu dengan lebih baik dibandingkan model Logistic Regression.**

Metrik	LR	LightGBM	Selisih
Specificity	0.8178	0.8222	+0.0044
FPR (salah tolak)	0.1822	0.1778	-0.0044
Recall	0.5112	0.5219	+0.0107
AUC-ROC	0.7428	0.7538	+0.0110

	LR	LightGBM
Klien MAMPU BAYAR disetujui	46,235 (81.78%)	46,483 (82.22%)
Klien MAMPU BAYAR salah tolak	10,303 (18.22%)	10,055 (17.78%)
Defaulter BENAR ditolak	2,538 (51.12%)	2,591 (52.19%)
Defaulter LOLOS disetujui	2,427 (48.88%)	2,374 (47.81%)



Preprocessing on Data Test

Setelah melakukan data preprocessing dan melatih model pada data train dan data validation, tahapan terakhir adalah melakukan pengujian pada data test, yaitu data yang belum pernah diprediksi oleh model.

Data test berikut memiliki file yang telah terpisah sejak awal dari data train dan validation dengan format dan isi data yang serupa, sehingga hanya perlu melakukan pemrosesan data kembali seperti yang telah dilakukan sebelumnya lalu mengujinya dengan model yang telah di-tuning dan digunakan sebelumnya.

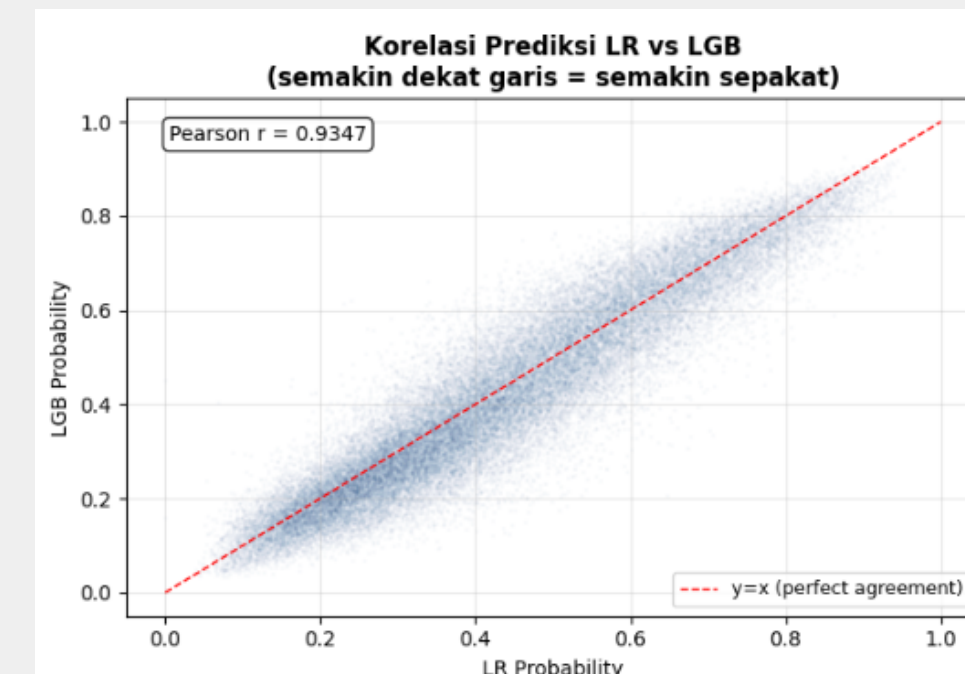
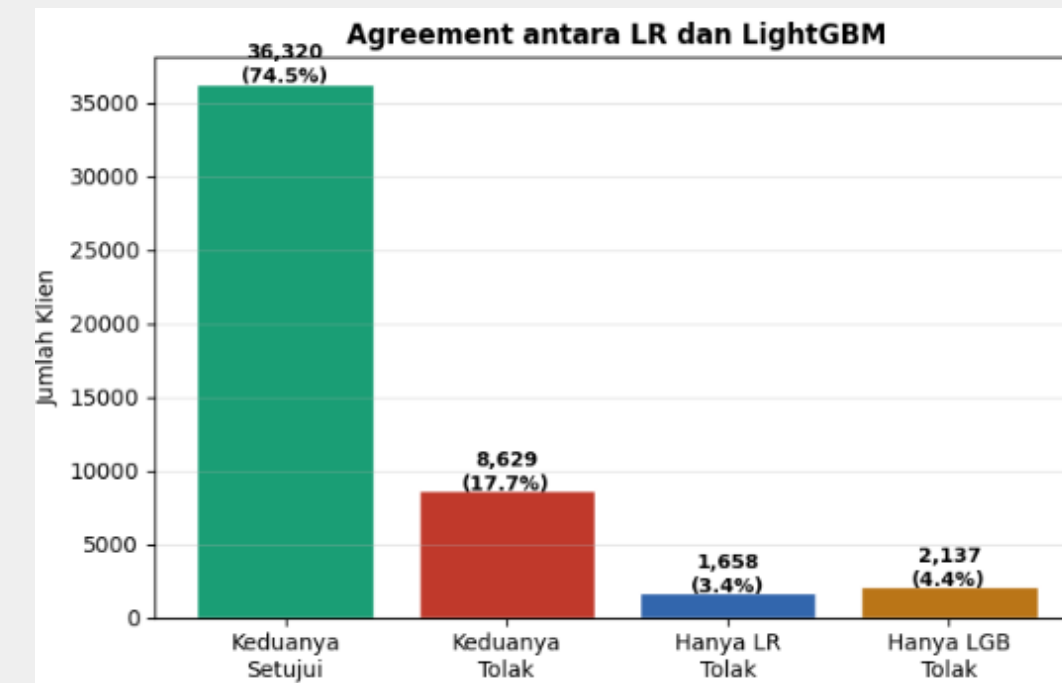
```
# =====  
# STEP 1 – LOAD & CEK DATA TEST  
# =====  
df_test = pd.read_csv(PATH + 'application_test.csv')  
sk_id = df_test['SK_ID_CURR'].copy()  
  
print(f'Shape data test : {df_test.shape}')  
print(f'Kolom TARGET ada : {"TARGET" in df_test.columns}')  
print(f'Jumlah SK_ID_CURR : {sk_id.nunique():,}')  
  
# =====  
# STEP 2 – PREPROCESSING (identik dengan train)  
# =====  
  
# — 2A. Drop kolom  
missing_pct = df_test.isnull().mean()  
drop_missing = [c for c in missing_pct[missing_pct > 0.5].index  
                 if c != 'EXT_SOURCE_1' and c in df_test.columns]  
  
drop_redundant = [c for c in df_test.columns  
                  if 'AVG' in c or '_MODE' in c or '_MEDI' in c]  
  
drop_flag_doc = [c for c in df_test.columns if 'FLAG_DOCUMENT' in c]  
drop_lowcorr = ['SK_ID_CURR', 'FLAG_CONT_MOBILE', 'FLAG_MOBIL', 'FLAG_EMAIL',  
                 'AMT_REQ_CREDIT_BUREAU_DAY', 'AMT_REQ_CREDIT_BUREAU_QRT',  
                 'LIVE_REGION_NOT_WORK_REGION', 'REG_REGION_NOT_LIVE_REGION',  
                 'REG_REGION_NOT_WORK_REGION']  
  
drop_highcorr = ['DAYS_EMPLOYED', 'OBS_60_CNT_SOCIAL_CIRCLE', 'AMT_CREDIT',  
                 'REGION_RATING_CLIENT', 'CNT_FAM_MEMBERS',  
                 'DEF_60_CNT_SOCIAL_CIRCLE']  
  
drop_insignificant= ['FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_TYPE_SUITE',  
                    'WEEKDAY_APPR_PROCESS_START', 'CNT_CHILDREN',  
                    'REGION_POPULATION_RELATIVE', 'REG_CITY_NOT_WORK_CITY',  
                    'LIVE_CITY_NOT_WORK_CITY', 'OBS_30_CNT_SOCIAL_CIRCLE',  
                    'AMT_REQ_CREDIT_BUREAU_MON']  
  
all_drop = set(drop_missing + drop_redundant + drop_flag_doc +  
               drop_lowcorr + drop_highcorr + drop_insignificant)  
df_test.drop(columns=[c for c in all_drop if c in df_test.columns],  
             inplace=True)  
print(f'\nShape setelah drop : {df_test.shape}')  
  
# — 2B. Handle Missing Values  
# Imputasi pakai median TRAIN – bukan median test!  
df_test['OCCUPATION_TYPE'] = df_test['OCCUPATION_TYPE'].fillna('Unknown')  
df_test['ORGANIZATION_TYPE'] = df_test['ORGANIZATION_TYPE'].replace('XNA', 'No_Organization')
```

```
# Flag SEBELUM imputasi  
df_test['EXT_SOURCE_3_MISSING'] = df_test['EXT_SOURCE_3'].isnull().astype(int)  
df_test['IS_NO_BUREAU_ENQUIRY'] = df_test['AMT_REQ_CREDIT_BUREAU_YEAR'].isnull().astype(int) \\\n    if 'AMT_REQ_CREDIT_BUREAU_YEAR' in df_test.columns else 0  
  
# Imputasi dengan median dari train  
TRAIN_MEDIAN = TRAIN_MEDIAN  
for col, val in TRAIN_MEDIAN.items():  
    if col in df_test.columns:  
        df_test[col] = df_test[col].fillna(val)  
  
for col in ['AMT_REQ_CREDIT_BUREAU_YEAR',  
            'DEF_30_CNT_SOCIAL_CIRCLE',  
            'OBS_30_CNT_SOCIAL_CIRCLE']:  
    if col in df_test.columns:  
        df_test[col] = df_test[col].fillna(0)  
  
# — 2C. Feature Engineering  
eps = 1e-9  
  
df_test['AGE_YEARS'] = (-df_test['DAYS_BIRTH'] / 365).round(1)  
df_test['REGISTRATION_YEARS'] = (-df_test['DAYS_REGISTRATION'] / 365).round(1)  
df_test['ID_PUBLISH_YEARS'] = (-df_test['DAYS_ID_PUBLISH'] / 365).round(1)  
df_test['PHONE_CHANGE_YEARS'] = (-df_test['DAYS_LAST_PHONE_CHANGE'] / 365).round(1)  
df_test.drop(columns=['DAYS_BIRTH', 'DAYS_REGISTRATION',  
                     'DAYS_ID_PUBLISH', 'DAYS_LAST_PHONE_CHANGE'],  
             inplace=True, errors='ignore')  
  
df_test['ANNUITY_INCOME_RATIO'] = df_test['AMT_ANNUITY'] / (df_test['AMT_INCOME_TOTAL'] + eps)  
df_test['GOODS_INCOME_RATIO'] = df_test['AMT_GOODS_PRICE'] / (df_test['AMT_INCOME_TOTAL'] + eps)  
df_test['ANNUITY_GOODS_RATIO'] = df_test['AMT_ANNUITY'] / (df_test['AMT_GOODS_PRICE'] + eps)  
  
df_test['EXT_SOURCE_MEAN'] = df_test[['EXT_SOURCE_2', 'EXT_SOURCE_3']].mean(axis=1)  
df_test['EXT_SOURCE_MIN'] = df_test[['EXT_SOURCE_2', 'EXT_SOURCE_3']].min(axis=1)  
df_test['EXT_SOURCE_PROD'] = df_test['EXT_SOURCE_2'] * df_test['EXT_SOURCE_3']  
  
df_test['ADDRESS_MISMATCH_SCORE'] = df_test['REG_CITY_NOT_LIVE_CITY'].astype(int)  
df_test['RECENTLY_CHANGED_ID'] = (df_test['ID_PUBLISH_YEARS'] < 1).astype(int) \\\n    if 'ID_PUBLISH_YEARS' in df_test.columns else 0  
df_test['RECENTLY_CHANGED_PHONE'] = (df_test['PHONE_CHANGE_YEARS'] < 1).astype(int) \\\n    if 'PHONE_CHANGE_YEARS' in df_test.columns else 0  
  
df_test['LOG_AMT_INCOME_TOTAL'] = np.log1p(df_test['AMT_INCOME_TOTAL'])  
df_test['LOG_AMT_ANNUITY'] = np.log1p(df_test['AMT_ANNUITY'])  
df_test['LOG_AMT_GOODS_PRICE'] = np.log1p(df_test['AMT_GOODS_PRICE'])  
df_test.drop(columns=['AMT_INCOME_TOTAL', 'AMT_ANNUITY', 'AMT_GOODS_PRICE'],  
             inplace=True, errors='ignore')  
  
# Outlier capping – nilai cap dari train  
cap_values = {  
    'DEF_30_CNT_SOCIAL_CIRCLE' : 4.0,  
    'AMT_REQ_CREDIT_BUREAU_YEAR': 8.0,  
    'ANNUITY_INCOME_RATIO' : 0.5,  
    'GOODS_INCOME_RATIO' : 3.5,  
    'ANNUITY_GOODS_RATIO' : 0.15,  
}  
for col, cap_val in cap_values.items():  
    if col in df_test.columns:  
        df_test[col] = df_test[col].clip(upper=cap_val)  
  
# — 2D. Encoding  
from sklearn.preprocessing import LabelEncoder  
  
cat_cols = df_test.select_dtypes('object').columns.tolist()  
binary_cats = [c for c in cat_cols if df_test[c].nunique() <= 2]  
nonbinary_cats = [c for c in cat_cols if df_test[c].nunique() > 2]
```

Model Prediction on Data Test

Berikut adalah hasil prediksi pada data test:

1. Baik model Logistic Regression maupun LightGBM sama-sama menerima dan menolak pengajuan kredit dari data klien yang sama berurut sebesar 36.320 klien (74.5%) dan 8.629 klien (17.7%)
2. Pada data test, LightGBM lebih banyak menolak klien, yang tidak ditolak oleh Logistic Regression, sebanyak 2.137 klien (4.4%)
3. Sebaliknya, Logistic Regression lebih sedikit menolak klien, yang diterima oleh LightGBM sebesar 1.658 klien (3.4%)
4. Pada grafik korelasi, kedua klien ini memiliki nilai korelasi Pearson r yang sangat tinggi, sebesar 0.9347. Artinya, kedua model memberikan skor risiko yang sangat mirip.
5. Sebaran titik terlihat lebih tebal pada area probabilitas 0.0-0.4 yang menandakan bahwa model ini lebih banyak memberikan skor rendah dibandingkan Logistic Regression



**Dokumentasi selengkapnya
dapat dibaca di [sini!](#)**

Thank you!



Rakamin
Academy

X

**HOME
CREDIT**
JADI BISA!